MÉTRICAS CK (FASE SÊNIOR)

QUESTÕES DE MÉTRICAS CK

ASSUNTO 1: WMC (Weighted Methods per Class) — Complexidade da Classe

Questão 1 — Conceitual (1 Ponto)
Pergunta: O que a métrica WMC (Weighted Methods per Class) mede em uma classe sob a perspectiva da suíte CK?
[ ] A) O número total de linhas físicas escritas dentro de cada método.
[x] B) A complexidade interna da classe por meio da contagem de seus métodos. (CORRETA)
[ ] C) A quantidade de classes filhas que herdam suas assinaturas.
[ ] D) O número de conexões ativas que a classe estabelece com a rede.
Explicação (Flashcard): O WMC é uma métrica estrutural que quantifica a complexidade de uma classe. Em sua abordagem padrão e mais utilizada, ela calcula simplesmente o número total de métodos declarados diretamente na classe.

Questão 2 — Vantagens e Desvantagens (2 Pontos)
Pergunta: Qual é o principal problema indicado por uma classe que apresenta um indicador WMC excessivamente alto?
[x] A) A classe assumiu responsabilidades demais, violando o Princípio de Responsabilidade Única (SRP). (CORRETA)
[ ] B) A árvore de herança do sistema ficou rasa demais, impedindo o reuso.
[ ] C) O arquivo foi totalmente ignorado pelas ferramentas de testes de cobertura.
[ ] D) O acoplamento da classe com bibliotecas externas caiu para zero.
Explicação (Flashcard): Um valor de WMC muito alto é um forte indicador de que a classe está fazendo tarefas demais e centralizando a lógica do sistema. Isso viola o SRP, tornando o código difícil de compreender, testar e manter.

Questão 3 — Exemplo Prático de Código (3 Pontos)
Pergunta: Analise o código Java abaixo:

```
class Produto {
    private double preco;
    void atualizarPreco(double p) { this.preco = p; }
    void aplicarDesconto(double d) { this.preco -= d; }
    boolean estaDisponivel() { return this.preco > 0; }
}
```

Qual é o valor exato do WMC desta classe considerando a fórmula tradicional?
[ ] A) WMC = 1.
[ ] B) WMC = 2.
[x] C) WMC = 3. (CORRETA)
[ ] D) WMC = 4.

Explicação (Flashcard): Aplicando a fórmula tradicional (onde cada método possui peso unitário), o WMC consiste na contagem simples das funções locais. Como a classe possui exatamente 3 métodos declarados, o WMC é igual a 3.

Questão 4 — DESAFIO (3 Pontos)
Pergunta: Uma classe legada de faturamento possui um WMC de 42. Qual estratégia de refatoração resolve esse problema de forma elegante seguindo as boas práticas de design OO?
[ ] A) Agrupar os métodos em um único bloco condicional gigante para reduzir a contagem.
[x] B) Aplicar o SRP, quebrando a classe inflada em novas classes menores, coesas e focadas. (CORRETA)
[ ] C) Mover todos os atributos para um banco de dados global para eliminar as variáveis de classe.
[ ] D) Herdar essa classe em 42 subclasses diferentes para distribuir as chamadas.
Explicação (Flashcard): A presença de um WMC altíssimo (como 42) sinaliza uma "God Class" ou classe acumuladora. A refatoração correta é a decomposição da classe com base em suas responsabilidades, extraindo novas classes menores com escopos bem definidos.

ASSUNTO 2: DIT (Depth of Inheritance Tree) — Profundidade de Herança

Questão 1 — Conceitual (1 Ponto)
Pergunta: Como funciona a medição feita pela métrica DIT (Depth of Inheritance Tree)?
[x] A) Calcula a distância máxima de uma classe até a sua classe raiz ancestral na árvore de herança. (CORRETA)
[ ] B) Conta quantas classes herdam propriedades de uma mesma interface abstrata.
[ ] C) Mede a quantidade de métodos que foram sobrescritos com polimorfismo.
[ ] D) Avalia o tempo gasto pelo compilador para resolver referências genéricas.
Explicação (Flashcard): O DIT mede a profundidade da hierarquia de herança. O cálculo rastreia quantos níveis de herança existem entre a classe sob análise e a classe ancestral raiz (que fica no topo, no nível 0).

Questão 2 — Vantagens e Desvantagens (2 Pontos)
Pergunta: Qual é a consequência de projetarmos um sistema com hierarquias de herança extremamente profundas (DIT muito alto)?
[ ] A) Aumento obrigatório da taxa de acoplamento dinâmico entre objetos desconexos.
[ ] B) O código-fonte se torna impossível de ser compilado em ambientes locais.
[x] C) Dificulta o entendimento e a manutenção, pois o comportamento fica fragmentado em muitas classes. (CORRETA)
[ ] D) Elimina a possibilidade de reutilizar os métodos das classes ancestrais.
Explicação (Flashcard): Embora a herança favoreça a reutilização, um DIT excessivamente alto distribui os métodos e comportamentos em muitas camadas. Isso dificulta muito o entendimento global da lógica por parte dos desenvolvedores.

Questão 3 — Exemplo Prático de Código (3 Pontos)
Pergunta: Considere a estrutura declarada abaixo:

```
class Dispositivo { }
class Computador extends Dispositivo { }
class Notebook extends Computador { }
```

Qual é o valor de DIT para a classe Notebook?
[ ] A) DIT = 0.

[ ] B) DIT = 1.
[x] C) DIT = 2. (CORRETA)
[ ] D) DIT = 3.
Explicação (Flashcard): A classe ancestral raiz `Dispositivo` está no nível 0. `Computador` herda dela e fica no nível 1. Como `Notebook` estende `Computador`, sua profundidade na árvore é igual a 2.

Questão 4 — DESAFIO (3 Pontos)
Pergunta: Em um projeto corporativo de grande porte, constatou-se que a classe `GerenteSenior` possui um DIT igual a 6. Qual prática arquitetural de OO resolve esse acoplamento rígido de herança?
[x] A) Substituir a herança profunda por composição de objetos. (CORRETA)
[ ] B) Converter todos os métodos da cadeia para assinaturas privadas e sem parâmetros.
[ ] C) Unir todas as 6 classes ancestrais em uma única interface vazia.
[ ] D) Forçar o uso de herança múltipla usando linguagens como C++.
Explicação (Flashcard): Um DIT muito alto gera forte acoplamento e fragilidade. Em design de software, a melhor prática para suavizar essa hierarquia é preferir a composição à herança (Favor Composition over Inheritance), acoplando os componentes por meio de atributos e interfaces finas.

ASSUNTO 3: NOC (Number of Children) — Quantidade de Subclasses

Questão 1 — Conceitual (1 Ponto)
Pergunta: O que quantifica diretamente a métrica NOC (Number of Children) em um projeto?
[ ] A) O número total de variáveis declaradas dentro de um construtor.
[ ] B) A quantidade de chamadas externas recebidas por um método.
[x] C) O número de subclasses que herdam diretamente de uma classe pai. (CORRETA)
[ ] D) O total de métodos abstratos que devem ser implementados obrigatoriamente.
Explicação (Flashcard): O NOC conta a quantidade de subclasses imediatas (filhos de primeiro nível) que estendem de forma direta uma classe pai específica.

Questão 2 — Vantagens e Desvantagens (2 Pontos)
Pergunta: Qual é o principal risco técnico de alterarmos o código de uma classe pai que apresenta um indicador NOC muito elevado?
[x] A) A mudança pode propagar efeitos colaterais inesperados e quebrar muitas subclasses filhas. (CORRETA)
[ ] B) A complexidade ciclomática da própria classe pai será reduzida a zero.
[ ] C) O tempo gasto para compilar o arquivo passará a ser processado de forma lenta.
[ ] D) A classe pai perderá sua capacidade de ser instanciada diretamente no sistema.
Explicação (Flashcard): Um NOC alto mostra que a classe é uma abstração importante e altamente reutilizada. Contudo, mudanças em suas assinaturas ou lógicas são arriscadas porque impactam diretamente todos os seus filhos, exigindo testes de regressão exaustivos.

Questão 3 — Exemplo Prático de Código (3 Pontos)
Pergunta: Analise as declarações de classe abaixo:

```
class FormaGeometrica { }
class Circulo extends FormaGeometrica { }
class Quadrado extends FormaGeometrica { }
class Triangulo extends FormaGeometrica { }
```

Qual é o NOC correspondente da classe FormaGeometrica?
[ ] A) NOC = 1.
[ ] B) NOC = 2.
[x] C) NOC = 3. (CORRETA)
[ ] D) NOC = 4.
Explicação (Flashcard): Como as três classes (`Circulo`, `Quadrado` e `Triangulo`) herdam de forma direta da classe `FormaGeometrica`, ela possui exatamente 3 filhos imediatos, logo seu NOC é 3.

Questão 4 — DESAFIO (3 Pontos)
Pergunta: Uma classe base do núcleo do sistema possui um NOC de 14. O time precisa realizar uma refatoração em sua principal assinatura. Qual abordagem técnica garante a segurança dessa manutenção?
[ ] A) Apagar as subclasses para zerar o NOC antes de fazer as alterações.
[ ] B) Mesclar todos os filhos em um único arquivo de classe gigante.
[x] C) Aplicar testes automatizados de regressão cobrindo o comportamento de todas as subclasses. (CORRETA)
[ ] D) Trocar o modificador de acesso da classe pai de public para private.
Explicação (Flashcard): Classes com NOC alto demandam extrema cautela e cobertura de testes. Para refatorar com segurança, a equipe deve contar com uma bateria robusta de testes de regressão automatizados para validar que nenhuma das 14 subclasses foi quebrada pela alteração no pai.

ASSUNTO 4: CBO (Coupling Between Objects) — Acoplamento de Classes

Questão 1 — Conceitual (1 Ponto)
Pergunta: O que mede a métrica CBO (Coupling Between Objects) na suíte CK?
[ ] A) O número total de linhas lógicas ocupadas por variáveis locais.
[ ] B) A quantidade de loops e desvios condicionais aninhados no código.
[x] C) O nível de dependência que uma classe tem em relação a outras classes externas. (CORRETA)
[ ] D) A taxa de erros ocorridos na execução de testes integrados.
Explicação (Flashcard): O CBO quantifica o acoplamento avaliando quantas outras classes do sistema são utilizadas pela classe atual (por meio de herança, tipos de atributos, retornos ou declarações).

Questão 2 — Vantagens e Desvantagens (2 Pontos)
Pergunta: Por que manter o CBO baixo é considerado uma das regras de ouro do design orientado a objetos?
[ ] A) Porque força o uso de heranças profundas com DIT muito alto.
[x] B) Facilita a manutenção isolada, reduz o impacto de mudanças e simplifica os testes. (CORRETA)
[ ] C) Garante que a classe não necessite de nenhum comentário explicativo.
[ ] D) Permite que a classe acumule mais de dezenas de responsabilidades distintas.
Explicação (Flashcard): Classes com baixo acoplamento (CBO baixo) são independentes. Isso significa que podemos alterar ou testar a classe sem quebrar outros módulos, além de facilitar testes unitários rápidos sem depender de cenários de simulação (mocks) gigantes.

Questão 3 — Exemplo Prático de Código (3 Pontos)
Pergunta: Avalie a estrutura da classe abaixo:

```
class RelatorioService {
    private BancoDeDados db;
    private FormatadorPDF pdf;
    private GeradorGrafico grafico;
}
```

Qual é o CBO da classe RelatorioService considerando as classes externas declaradas?
[ ] A) CBO = 0.
[ ] B) CBO = 1.
[x] C) CBO = 3. (CORRETA)
[ ] D) CBO = 4.
Explicação (Flashcard): A classe `RelatorioService` depende diretamente da existência e estrutura de 3 classes externas (`BancoDeDados`, `FormatadorPDF` e `GeradorGrafico`) para definir seus atributos. Logo, seu CBO é 3.

Questão 4 — DESAFIO (3 Pontos)
Pergunta: Durante uma auditoria técnica, um analista descobriu que uma classe controladora de envio de e-mails possui um CBO de 15. Qual padrão de design reduz esse acoplamento excessivo de forma elegante?
[x] A) Introduzir injeção de dependência por meio de interfaces abstratas. (CORRETA)
[ ] B) Criar uma classe herdada para agrupar as 15 dependências de forma rígida.
[ ] C) Substituir todos os tipos de objetos por variáveis primitivas do tipo string.
[ ] D) Copiar os métodos de todas as 15 classes diretamente para dentro do controlador.
Explicação (Flashcard): CBO alto significa alto acoplamento concreto. Para desacoplar as classes, aplica-se o princípio de injeção de dependência baseado em interfaces (DIP - Dependency Inversion Principle). Isso substitui o acoplamento com classes concretas por abstrações estáveis, reduzindo drasticamente o CBO real.

ASSUNTO 5: RFC (Response For a Class) — Complexidade de Resposta

Questão 1 — Conceitual (1 Ponto)
Pergunta: Qual é o foco de análise de um indicador RFC (Response For a Class)?
[ ] A) Medir a profundidade física de aninhamento dos blocos lógicos.
[x] B) Mapear a quantidade de métodos que podem ser executados após um estímulo à classe. (CORRETA)
[ ] C) Contar quantas requisições HTTP a classe consegue responder por segundo.
[ ] D) Avaliar o tamanho total ocupado pelo código compilado na memória.
Explicação (Flashcard): O RFC mede a complexidade de resposta e o acoplamento de execução (dinâmico). Ele representa o conjunto de métodos que podem ser chamados no sistema a partir de uma mensagem recebida por um objeto da classe.

Questão 2 — Vantagens e Desvantagens (2 Pontos)
Pergunta: Qual é a principal dificuldade técnica encontrada ao dar manutenção em uma classe que apresenta RFC muito elevado?
[ ] A) A classe se torna inelegível para o uso de herança estrutural simples.
[ ] B) A contagem de linhas físicas (LOC) da classe é zerada de forma forçada.
[x] C) Fica extremamente difícil prever seu comportamento e projetar testes unitários eficazes. (CORRETA)
[ ] D) O compilador passa a ignorar os métodos locais declarados nela.
Explicação (Flashcard): Um RFC alto indica que a execução da classe dispara chamadas em

cascata por métodos de outras classes do sistema. Isso torna o comportamento difícil de rastrear e depurar, exigindo testes unitários complexos que precisam simular muitas chamadas externas.

Questão 3 — Exemplo Prático de Código (3 Pontos)
Pergunta: Analise o trecho de código abaixo:

```
class Checkout {
    void processar() {
        validarCarrinho(); // Método de outra classe
        calcularFrete(); // Método de outra classe
    }
}
```

Sabendo que `processar` é o único método declarado localmente em Checkout e que ele chama dois métodos externos, qual é o RFC de Checkout?
[ ] A) RFC = 1.
[ ] B) RFC = 2.
[x] C) RFC = 3. (CORRETA)
[ ] D) RFC = 4.
Explicação (Flashcard): O RFC é calculado somando os métodos locais da própria classe (1 método, que é `processar`) com a quantidade de métodos externos invocados por eles (2 métodos, `validarCarrinho` e `calcularFrete`). Portanto, $RFC = 1 + 2 = 3$.

Questão 4 — DESAFIO (3 Pontos)
Pergunta: Em testes unitários, por que uma classe com RFC elevado (por exemplo, RFC = 45) gera o que o mercado chama de ”pesadelo de mocks”?
[ ] A) Porque ela impede que o framework de testes seja executado no servidor local.
[ ] B) Porque ela zera o WMC e impede a leitura de atributos públicos.
[x] C) Porque o desenvolvedor precisará simular (mockar) dezenas de comportamentos externos para validar um único fluxo. (CORRETA)
[ ] D) Porque ela restringe os testes apenas à validação de assinaturas de herança de nível 0.
Explicação (Flashcard): Como a classe com RFC alto invoca muitos métodos externos de forma encadeada, o teste unitário isolado dessa classe exige que o desenvolvedor crie dublês de teste (mocks) para dezenas de métodos de outras classes para conseguir rodar e validar seu fluxo lógico interno.

ASSUNTO 6: LCOM (Lack of Cohesion in Methods) — Falta de Coesão nos Métodos

Questão 1 — Conceitual (1 Ponto)
Pergunta: O que a métrica LCOM (Lack of Cohesion in Methods) mede de forma estrutural?
[x] A) A ausência de coesão, calculando quantos métodos da classe não compartilham o uso de atributos comuns. (CORRETA)
[ ] B) A quantidade de parâmetros passados de forma incorreta nas funções.
[ ] C) O número de linhas lógicas redundantes que devem ser excluídas.
[ ] D) O total de dependências herdadas através de superclasses abstratas.
Explicação (Flashcard): O LCOM mede a falta de coesão (Lack of Cohesion). Ele analisa se os métodos da própria classe compartilham os seus atributos de instância. Um LCOM baixo indica alta coesão de dados e lógica.

Questão 2 — Vantagens e Desvantagens (2 Pontos)

Pergunta: Por que é desejável mantermos um LCOM baixo (próximo de zero) em todas as classes do nosso sistema?

[ ] A) Porque isso indica que a classe possui um acoplamento dinâmico muito alto (RFC alto).

[ ] B) Significa que o arquivo de código-fonte possui poucas quebras de linha física.

[x] C) Mostra que a classe é focada, altamente coesa e que seus métodos trabalham juntos. (CORRETA)

[ ] D) Garante que a árvore de herança (DIT) cresça sem limites de herança.

Explicação (Flashcard): Um LCOM baixo (ou igual a zero) significa pouca falta de coesão, ou seja, alta coesão. Isso indica que os métodos da classe trabalham em conjunto compartilhando os mesmos atributos, caracterizando um componente focado e bem estruturado.

Questão 3 — Exemplo Prático de Código (3 Pontos)

Pergunta: Avalie as duas classes abaixo:

```
Class A {
    int x;
    void m1() { x = 10; }
    void m2() { x = 20; }
}

Class B {
    int y; int z;
    void m3() { y = 5; }
    void m4() { z = 10; }
}
```

Qual das classes apresenta melhor coesão (menor LCOM)?

[x] A) A Classe A, pois seus métodos compartilham o atributo 'x'. (CORRETA)

[ ] B) A Classe B, pois possui mais métodos e atributos declarados.

[ ] C) Ambas possuem o mesmo LCOM por apresentarem 2 métodos cada.

[ ] D) Nenhuma possui LCOM calculável por não usarem herança de superclasses.

Explicação (Flashcard): Na Classe A, todos os métodos (`m1` e `m2`) compartilham e atuam sobre o mesmo atributo de instância (`x`), gerando alta coesão (LCOM baixo). Na Classe B, os métodos operam em variáveis totalmente distintas (`m3` usa `y` e `m4` usa `z`), demonstrando falta de coesão (LCOM alto).

Questão 4 — DESAFIO (3 Pontos)

Pergunta: Uma classe de controle de inventário possui LCOM alto. Ao inspecionar, você nota que 3 métodos só utilizam atributos de estoque e outros 3 métodos só utilizam atributos de fornecedores. Como refatorar essa estrutura?

[ ] A) Criar uma única variável global para unificar os conceitos de fornecedor e estoque.

[x] B) Dividir a classe em duas novas classes menores e focadas: uma de Estoque e outra de Fornecedores. (CORRETA)

[ ] C) Forçar todos os métodos a utilizarem todos os atributos mesmo sem necessidade real.

[ ] D) Herdar a classe de inventário em uma cadeia de herança com DIT maior que 4.

Explicação (Flashcard): Um LCOM alto indica que a classe abriga lógicas e domínios de dados desconexos, agindo como duas ou mais entidades agrupadas incorretamente. A refatoração ideal é a separação das lógicas, criando duas classes pequenas e altamente coesas com propósitos bem delimitados.